

PREGUNTAS – LAB3

Pregunta 1

Cuando se arranca el servidor por primera vez y no existe ningún modelo previo en la carpeta `models/`, el sistema crea automáticamente un modelo base utilizando el dataset Iris completo (proceso de *bootstrap*).

Si detuviéramos el servidor, borraríamos la carpeta `models/` y volviéramos a arrancarlo, en principio deberíamos obtener el mismo modelo con la misma accuracy, siempre que:

- Se utilice exactamente el mismo dataset.
- El algoritmo y sus parámetros sean los mismos.
- No haya ningún elemento aleatorio sin control.

Sin embargo, si no se fija una semilla aleatoria (*random_state*), el modelo podría variar ligeramente entre ejecuciones, aunque las diferencias normalmente serían pequeñas.

Por tanto, la reproducibilidad depende de si el código fija o no una semilla aleatoria. Si el parámetro `random_state` está definido en el entrenamiento del modelo, entonces sí obtendríamos exactamente el mismo modelo y la misma accuracy. Si no está definido, podría haber pequeñas variaciones entre ejecuciones.

Pregunta 2

Si enviáramos 10 muestras perfectamente etiquetadas pero todas de la clase *setosa*, el modelo no se entrenaría correctamente.

Paso a paso, lo que ocurriría sería lo siguiente:

1. El endpoint `/train` comprobaría primero que hay al menos 5 muestras. En este caso sí se cumple, porque enviamos 10.
2. Después comprobaría que existen al menos 2 clases distintas en el dataset de entrenamiento. Aquí no se cumple, porque todas las muestras son de la misma clase (*setosa*).
3. Al no cumplirse esta condición, la API devolvería un error indicando que el dataset debe contener al menos dos clases distintas.

Por tanto:

- El modelo no se entrenaría.
- No se generaría una nueva versión.
- No se activaría ningún modelo nuevo.

Esta restricción tiene sentido porque un modelo de clasificación necesita aprender a distinguir entre varias clases. Si solo recibe ejemplos de una única clase, no puede aprender ninguna frontera de decisión ni generalizar correctamente. Básicamente, solo “aprendería” que todo es *setosa*, lo cual no sería útil en un problema multiclase como Iris.

Pregunta 3

Cuando ejecuto la demo con `--reset`, observo que en el historial final aparecen todas las versiones generadas, incluyendo el modelo entrenado con muestras ruidosas, aunque no haya sido activado.

Creo que este diseño es intencional y tiene mucho sentido. Aunque un modelo sea rechazado, sigue formando parte del proceso de entrenamiento y deja constancia de que se intentó crear una nueva versión.

Registrar también los modelos rechazados aporta información muy valiosa:

- Permite ver qué intentos se hicieron y por qué fallaron.
- Muestra cómo evolucionó el rendimiento del modelo a lo largo del tiempo.
- Ayuda a analizar si el problema fue la calidad de los datos, el tamaño del dataset o un descenso real en el rendimiento.

Si solo guardáramos los modelos activados, perderíamos información importante sobre decisiones pasadas. No sabríamos que existió un intento con datos ruidosos ni podríamos analizar por qué fue rechazado.

En un entorno real de producción, este historial completo es clave para entender el comportamiento del sistema y para justificar decisiones técnicas, especialmente si en algún momento el rendimiento empeora.

Pregunta 4

El gate de calidad utiliza la condición $\text{accuracy_nuevo} \geq \text{accuracy_anterior}$, lo que significa que un modelo nuevo se activa incluso si obtiene exactamente la misma accuracy que el anterior.

En mi opinión, esta regla tiene sentido en muchos casos. En la práctica, cuando el dataset va creciendo con muestras de buena calidad, puede llegar un punto en el que la accuracy se estabiliza cerca de su máximo. A partir de ahí, aunque añadamos datos útiles, la métrica puede no aumentar más porque el modelo ya está clasificando casi todo correctamente.

En ese escenario, si utilizáramos una condición estricta ($>$), el sistema no activaría nuevas versiones aunque el modelo esté entrenado con más datos y potencialmente sea más robusto. Es decir, podría estar aprendiendo mejor la estructura real del problema aunque la accuracy no suba.

Que dos versiones tengan la misma accuracy no significa necesariamente que sean idénticas en calidad. Puede que:

- Una esté entrenada con más datos.
- Una generalice mejor en casos no vistos.
- Una sea más estable frente a ruido.

Si usáramos $>$ en lugar de $\geq$, podríamos bloquear mejoras que no se reflejan inmediatamente en la métrica utilizada. Eso podría hacer que el sistema se quedara “congelado” en una versión antigua aunque el entrenamiento siga mejorando el modelo de fondo.

Por otro lado, usar $\geq$ también tiene riesgos: podríamos activar modelos que realmente no aportan mejora real, solo mantienen el mismo rendimiento.

En conclusión, creo que usar $\geq$ es una decisión razonable en este laboratorio porque permite que el modelo evolucione cuando el dataset crece y evita bloquear versiones potencialmente más robustas. En un entorno real, probablemente combinaría esta regla con más métricas (por ejemplo, precisión por clase o validaciones más amplias) para tomar decisiones más completas.